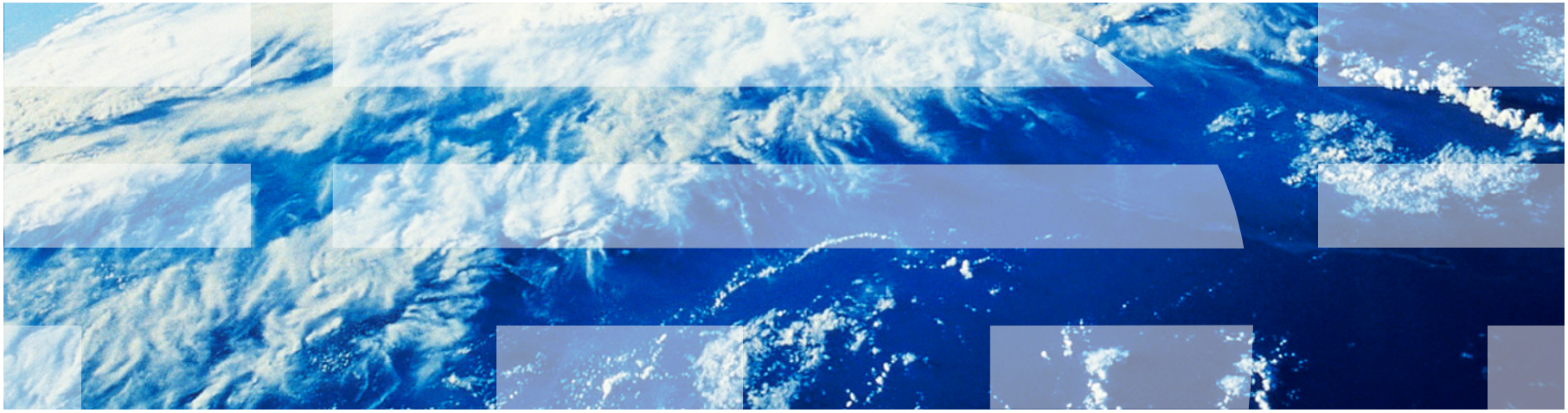
Borrowed from Shiva Shivakumar

Computer Systems for Data Science
Topic 3

Transactions and OLTP Databases



DBMS techniques and algorithms

- Write-ahead logging — Provides A and D in ACID
- Serializability — Whether I happens in ACID
- 2-Phase Locking — Help C and I in ACID
- Indexing — Faster access
- Bloom filters — Reduce unnecessary access attempts
- Caching — Reduce work duplication

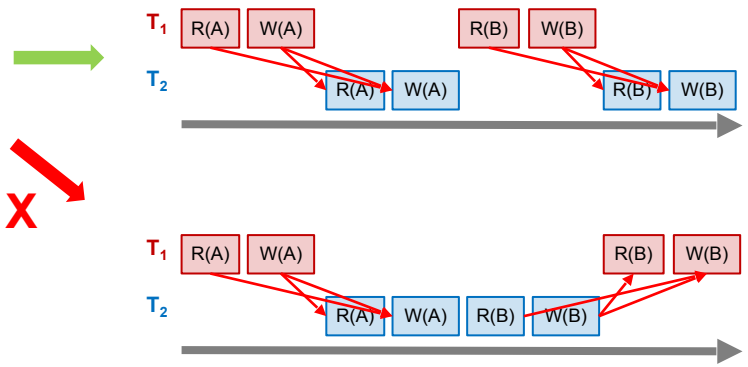
Anomaly terminologies
All of these are not allowed by serializability

- **Lost updates** (write-write conflict)
  - Two users update the same row at the same time, and one of their updates is forever lost without being committed
- **Dirty reads** (write-read conflict)
  - User reads an update that was never committed
    - E.g., Tx updates but then aborts, Ty reads the update even though it wasn't committed
- **Unrepeatable reads** (read-write conflict)
  - User reads two different values for same row within the same transaction
    - E.g., Tx reads value A=0, Ty updates value A=1 and commits, Tx reads value A again, reads A=1
- **Phantom reads** (read-write conflict)
  - User reads same set of rows twice, but new rows are inserted/removed to/from that set by another committed transaction in between

What can we say about “good” vs. “bad” conflict graphs?

Serial Schedule:

Interleaved Schedules:



Example with 5 transactions

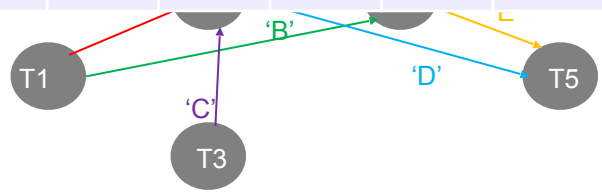
Schedule S1

Good or Bad schedule?
Conflict serializable?

	w1(A)	r2(A)	w1(B)	w3(C)	r2(C)	r4(B)	w2(D)	w4(E)	r5(D)	w5(E)
T1	w1(A)		w1(B)							
T2		r2(A)			R2(C)		w2(D)			
T3				w3(C)						
T4						r4(B)		w4(E)		
T5									r5(D)	w5(E)

Step1
Find conflicts
(RW, WW, WR)

Step2
Build Conflict graph
Acyclic?



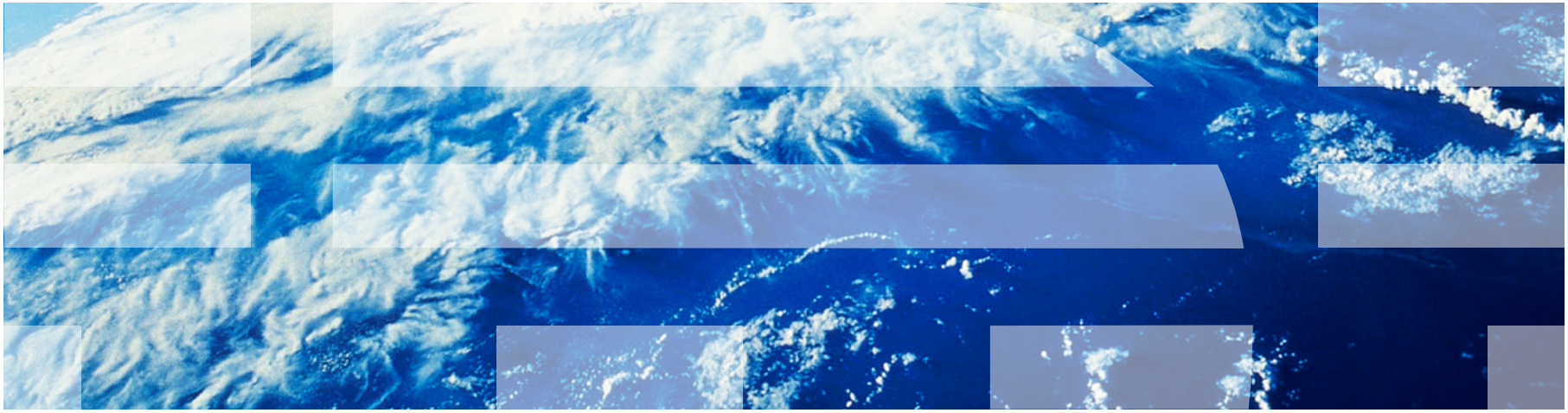
Acyclic
⇒ Conflict serializable!
⇒ Serializable

Step3
Example serial schedule
Conflict Equiv to S1

S2

T3	T1	T1	T4	T4	T2	T2	T2	T5	T5
w3(C)	w1(A)	w1(B)	r4(B)	w4(E)	r2(A)	r2(C)	w2(D)	r5(D)	w5(E)

2PL: A Simple Locking Algorithm



Strict Two-Phase Locking (2PL)

- Algorithm: *strict two-phase locking* - as a way to deal with concurrency
    - Guarantees conflict serializability
    - (if it completes- see upcoming…)

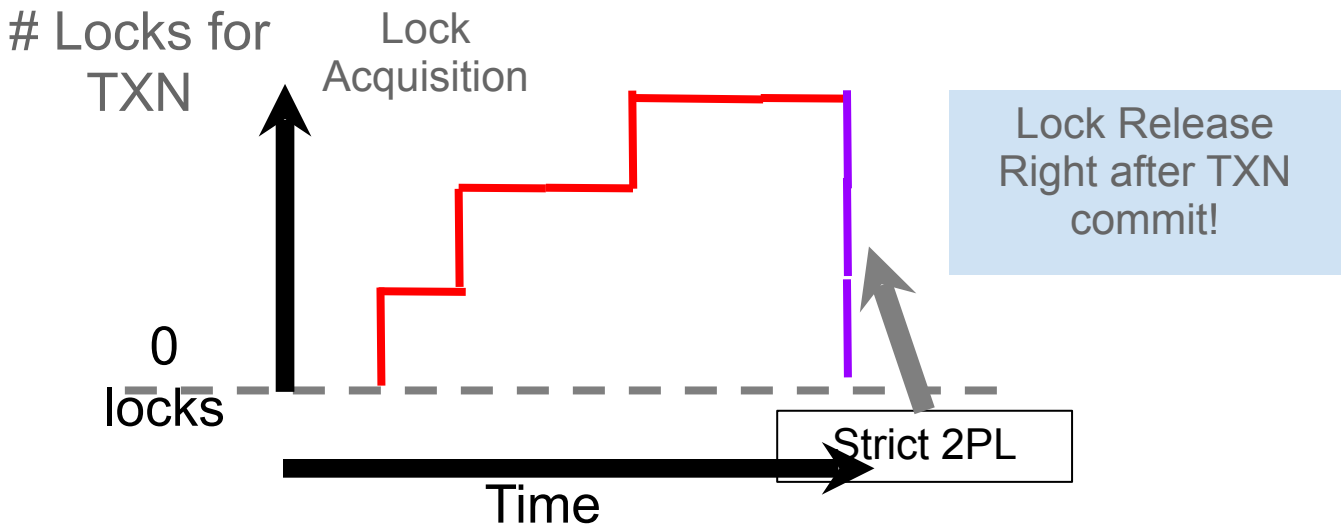
- Also (*conceptually*) straightforward to implement, and transparent to the user!

Strict Two-phase Locking (2PL) Protocol

TXNs obtain:

- An **X (*exclusive*) lock** on object before **writing**.
  - If a TXN holds, no other TXN can get a lock (S or X) on that object.
- An **S (*shared*) lock** on object before **reading**
  - If a TXN holds, no other TXN can get *an X lock* on that object
- All locks held by a TXN are released when TXN completes.

Picture of 2-Phase Locking (2PL)



2PL: A transaction can not request additional locks once it releases any locks. Thus, there is a “growing phase” followed by a “shrinking phase”.

Strict 2PL: Release locks right after COMMIT (COMMIT Record flushed) or ABORT

Strict 2PL

If a schedule follows strict 2PL, it is **conflict serializable**…

- …and thus serializable
- …and we get isolation & consistency!

Popular implementation

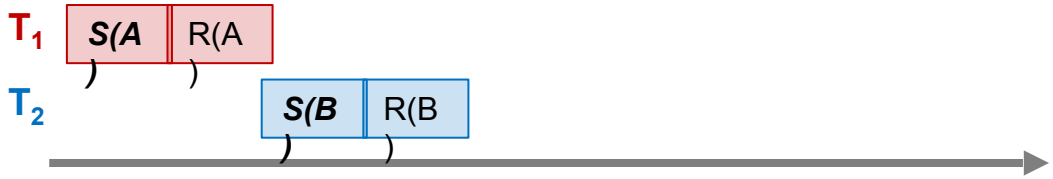
- Simple !
- Produces subset of *all* conflict serializable schedules
- There are MANY more complex LOCKING schemes

- One key, subtle problem (next)

Deadlock Detection

First, T_1 requests a shared lock on A to read from it

Deadlock Detection



Next, T_2 requests a shared lock on B to read from it